

FVG-detektor

Syfte

Syftet med detta arbete var att undersöka hur Python kan användas för att identifiera mönster i tidsseriedata, med Fair Value Gaps (FVG) i prisdata som konkret exempel. Målet var att bygga en mindre, fungerande lösning som automatiskt hittar dessa mönster i historisk data.

Område

Området för fördjupningen är bearbetning av tidsseriedata i Python, specifikt mönsterigenkänning genom att jämföra värden mellan datapunkter i en följd. Detta är relevant för Data Science-yrkesrollen eftersom tidsseriedata förekommer inom många branscher (finans, väder, sensordata) och förmågan att programmera regelbaserad mönsterigenkänning är en grundläggande färdighet innan man går vidare till mer avancerade metoder som maskininlärning.

Viktiga begrepp

Tidsserie: data ordnad i tidsföljd, t.ex. prisvärden per timme.

Ljus / candle: en datapunkt som visar öppnings-, stängnings-, högsta- och lägstavärde för en tidsperiod.

Fair Value Gap (FVG): ett mönster bestående av tre på varandra följande ljus, där det uppstår ett prisintervall (gap) där ingen handel skett, identifieras genom att jämföra high/low-värden mellan candle 1 och candle 3.

Genomförande

Lösningen byggdes i en Jupyter-notebook med Python. Biblioteket yfinance användes för att hämta historisk candle-data (timupplösning) för olika finansiella instrument, t.ex. för guld. Två funktioner skrevs: `bullish_fvg` och `bearish_fvg`, som loopar igenom candle-datan och jämför high/low-värden mellan candles för att identifiera gap. Det viktigaste tekniska valet var att hålla logiken enkel och regelbaserad för att fokusera på grundläggande mönsterigenkänning i tidsseriedata.

Resultat

Lösningen kan identifiera både bullish och bearish FVG i historisk candle-data, och ett hittat FVG kontrollerades visuellt mot en prisgraf. Testet på guld (5 dagar, timupplösning) hittade vid

körningstillfället 10 bullish och 5 bearish FVG. Eftersom koden hämtar data för de senaste 5 dagarna kommer man att få olika antal FVG vid körning på olika dagar. Samma funktioner testades även på EURUSD utan att koden behövde ändras (förutom vilket instrument som hämtades), vilket visar att lösningen är generell och återanvändbar för olika typer av prisdata. Varje FVG skrivs ut med datum och prisintervall, vilket gör resultatet begripligt och möjligt att verifiera manuellt mot en prisgraf.

```
fvg_lista = bullish_fvg(data)
for datum, gap_start, gap_end in fvg_lista:
    print("Bullish FVG:", datum, "gap:", gap_start, "-", gap_end)
```

✓ 0.0s

```
Bullish FVG: 2026-09-17 02:00:00-04:00 gap: [4337.5] - [4342.29980469]
Bullish FVG: 2026-09-17 03:00:00-04:00 gap: [4343.70019531] - [4347.79980469]
Bullish FVG: 2026-09-17 07:00:00-04:00 gap: [4358.5] - [4362.89990234]
Bullish FVG: 2026-09-17 08:00:00-04:00 gap: [4364.70019531] - [4383.79980469]
Bullish FVG: 2026-09-17 09:00:00-04:00 gap: [4393.5] - [4395.89990234]
Bullish FVG: 2026-09-18 01:00:00-04:00 gap: [4399.5] - [4399.79980469]
Bullish FVG: 2026-09-18 02:00:00-04:00 gap: [4406.89990234] - [4416.70019531]
Bullish FVG: 2026-09-18 03:00:00-04:00 gap: [4418.10009766] - [4429.]
Bullish FVG: 2026-09-18 13:00:00-04:00 gap: [4396.60009766] - [4419.89990234]
Bullish FVG: 2026-09-21 19:00:00-04:00 gap: [4383.29980469] - [4395.5]
```

Begränsningar och möjliga förbättringar

Lösningen identifierar endast mönstret i historisk data, den gör inga förutsägelser om framtida FVG, eftersom en FVG per definition bara kan bekräftas efter att tredje candelstängten stängts.

Lösningen tar inte hänsyn till hur ofta en FVG faktiskt fylls igen. En möjlig förbättring skulle vara att lägga till sådan statistik, eller att visualisera alla FVG direkt på en prisgraf istället för bara i text.

Valideringen var begränsad till en visuell kontroll av ett FVG. En mer systematisk kontroll skulle vara ett naturligt nästa steg.

En annan begränsning är att koden inte tar i beaktande hur små FVG som skapas. I praktiken skulle små FVG antingen inte anses som godkända eller att de fylls igen väldigt snabbt. Så nästa steg skulle kunna vara att lägga in en begränsning på hur stort ett FVG måste vara för att vara giltigt eller att inte ta med det om det stängs väldigt kort efter att det bildades.

Data hämtad via yfinance hade enstaka luckor (saknade timmar), vilket är en begränsning i datakällan. Det behöver undersökas vidare för att se vad det beror på och om det påverkar analysen.

Koppling till yrkesrollen

Tekniken som används, att loopa igenom tidsseriedata och jämföra värden för att hitta mönster, är en grundläggande färdighet för en Data Scientist eller dataanalytiker. Samma princip används vid t.ex. avvikelsedetektion i sensordata, identifiering av trender i försäljningsdata, eller förberedande databearbetning inför maskininlärning. Förmågan att bygga en tydlig, förklarbar regelbaserad lösning innan man går vidare till mer avancerade metoder är också relevant, eftersom enkla och transparenta modeller ofta föredras i praktiken när de räcker till.

Självreflektion

Jag har lärt mig att det inte är svårare att importera extern data än att importera t.ex. numpy eller pandas. Det som tog mest tid var att omsätta definitionen av en FVG till ett konkret jämförelsevillkor i kod. Det tekniska val jag är mest nöjd med är att skriva FVG-logiken som återanvändbara funktioner, vilket gjorde att samma kod kunde testas på flera instrument utan ändring. Om jag började om hade jag lagt mer tid på att visualisera och validera fler av de hittade FVG, inte bara ett exempel. Ett naturligt nästa steg skulle vara att lägga in en begränsning för hur stora FVG måste vara för att vara godkända.

Jag bedömer att arbetet motsvarar G. Jag uppfyller kraven för G, eftersom jag har byggt en fungerande lösning inom ett avgränsat område och kan förklara hur den fungerar.

Källor

<https://pypi.org/project/yfinance/>

Ultimate Guide to Trading Fair Value Gaps, NinjaTrader (för FVG-definitionen):

<https://ninjatrade.com/futures/blogs/fair-value-gaps/>